Beginner Trading Assistant Using RBR and CBR

Knowledge Representation and Reasoning

Ezekiel M. Ibasco | AN42

Knowledge Base and Use Case

The knowledge base uses three categories: market, investor, and portfolio. Its facts include price, moving average, RSI, trend, volatility, risk tolerance, experience, emergency-fund status, ownership, allocation, and available cash. The use case is a beginner seeking a simple educational interpretation of a manually entered market snapshot. The assistant does not access live prices, execute trades, or provide financial advice.

Rule Based Reasoning

Six IF-THEN functions examine trend confirmation, RSI, volatility and risk compatibility, portfolio concentration, available cash and emergency savings, and beginner position sizing. The inference engine checks every rule and prints every applicable action.

Case Based Reasoning

Four past cases contain problem features and solutions. Retrieval uses weighted exact matching: trend 30 percent, RSI band 25 percent, volatility 20 percent, risk tolerance 15 percent, and experience 10 percent. The system reuses the closest solution, permits manual revision, and retains the new case.

Comparative Reflection

RBR was easier to implement because each condition and action could be written directly as an IF-THEN rule. CBR is more flexible for unstructured problems because it can reuse experience even without an exact match. CBR can adapt more naturally to continuously changing data, although its case base must be reviewed. RBR fits compliance checks, eligibility screening, and safety alerts. CBR fits troubleshooting, support, case review, and recommendations. Here, RBR explains triggered limits clearly, while CBR suggests an action from the most similar past market situation.

Complete Python Program

```
# BEGINNER TRADING ASSISTANT
# Educational demonstration only - not financial advice.

from copy import deepcopy

# -------------------- PART 1: KNOWLEDGE REPRESENTATION --------------------
# Three categories hold the facts used by both reasoning systems.
facts = {
    'market': {
        'symbol': 'DEMO',
        'current_price': 105.0,
        'moving_average': 100.0,
        'rsi': 42,
        'trend': 'up',
        'volatility': 'medium',
    },
    'investor': {
        'risk_tolerance': 'low',
        'experience': 'beginner',
        'has_emergency_fund': True,
    },
    'portfolio': {
        'owns_asset': False,
        'allocation_percent': 0,
        'cash_available': 5000.0,
    },
}

# -------------------- PART 2: RULE-BASED REASONING --------------------
def rule_positive_trend(kb):
    market = kb['market']
    if market['trend'] == 'up' and market['current_price'] > market['moving_average']:
        return 'Positive trend detected: add the asset to a watchlist; do not rush to buy.'

def rule_rsi(kb):
    rsi = kb['market']['rsi']
    if rsi >= 70:
        return 'RSI is overbought: wait and avoid opening a new position.'
    if rsi <= 30:
        return 'RSI is oversold: investigate further before considering a small position.'
    return 'RSI is neutral: use other evidence before making a decision.'

def rule_risk_and_volatility(kb):
    if kb['investor']['risk_tolerance'] == 'low' and kb['market']['volatility'] == 'high':
        return 'Risk warning: high volatility does not match a low-risk profile.'

def rule_portfolio_limit(kb):
    if kb['portfolio']['allocation_percent'] >= 20:
        return 'Diversification warning: do not add because this asset is already 20% or more of the
portfolio.'

def rule_cash_and_safety(kb):
    if not kb['investor']['has_emergency_fund']:
        return 'Safety warning: build an emergency fund before considering a trade.'
    if kb['portfolio']['cash_available'] <= 0:
        return 'No available cash: do not borrow money to make this trade.'

def rule_beginner_size(kb):
    if kb['investor']['experience'] == 'beginner':
        return 'Beginner rule: if proceeding after research, use only a small amount and define an exit
plan.'

RBR_RULES = [
    rule_positive_trend,
    rule_rsi,
    rule_risk_and_volatility,
    rule_portfolio_limit,
    rule_cash_and_safety,
    rule_beginner_size,
]

def run_inference_engine(knowledge_base):
    actions = []
    for rule in RBR_RULES:
        action = rule(knowledge_base)
        if action:
            actions.append(action)
    return actions
```

```
# -------------------- PART 3: CASE-BASED REASONING --------------------
case_base = [
    {
        'case_id': 'TC-001',
        'problem': {'trend': 'up', 'rsi_band': 'neutral', 'volatility': 'medium', 'risk_tolerance':
'low', 'experience': 'beginner'},
        'solution': 'Watch the asset and, only after more research, consider a very small staged
purchase.',
    },
    {
        'case_id': 'TC-002',
        'problem': {'trend': 'down', 'rsi_band': 'oversold', 'volatility': 'high', 'risk_tolerance':
'medium', 'experience': 'intermediate'},
        'solution': 'Wait for trend confirmation because an oversold reading can remain weak during a
decline.',
    },
    {
        'case_id': 'TC-003',
        'problem': {'trend': 'up', 'rsi_band': 'overbought', 'volatility': 'high', 'risk_tolerance':
'low', 'experience': 'beginner'},
        'solution': 'Avoid entering while momentum is overbought and volatility exceeds the investor
profile.',
    },
    {
        'case_id': 'TC-004',
        'problem': {'trend': 'sideways', 'rsi_band': 'neutral', 'volatility': 'low', 'risk_tolerance':
'low', 'experience': 'beginner'},
        'solution': 'Continue observing and practice with a paper-trading plan before committing money.
',
    },
]

SIMILARITY_WEIGHTS = {
    'trend': 0.30,
    'rsi_band': 0.25,
    'volatility': 0.20,
    'risk_tolerance': 0.15,
    'experience': 0.10,
}

def get_rsi_band(rsi):
    if rsi <= 30:
        return 'oversold'
    if rsi >= 70:
        return 'overbought'
    return 'neutral'

def build_new_problem(knowledge_base):
    return {
        'trend': knowledge_base['market']['trend'],
        'rsi_band': get_rsi_band(knowledge_base['market']['rsi']),
        'volatility': knowledge_base['market']['volatility'],
        'risk_tolerance': knowledge_base['investor']['risk_tolerance'],
        'experience': knowledge_base['investor']['experience'],
    }

def calculate_similarity(new_problem, old_problem):
    return sum(
        weight
        for feature, weight in SIMILARITY_WEIGHTS.items()
        if new_problem[feature] == old_problem[feature]
    )

def retrieve_most_similar(case_base, new_problem):
    ranked = []
    for case in case_base:
        score = calculate_similarity(new_problem, case['problem'])
        ranked.append((score, case))
    ranked.sort(key=lambda item: item[0], reverse=True)
    return ranked

def reuse_solution(retrieved_case):
    return retrieved_case['solution']

def revise_solution(reused_solution, manual_revision=''):
    # Leave manual_revision empty to accept the retrieved solution unchanged.
    return manual_revision.strip() or reused_solution

def retain_case(case_base, problem, solution):
    new_id = f"TC-{len(case_base) + 1:03d}"
    case_base.append({'case_id': new_id, 'problem': deepcopy(problem), 'solution': solution})
    return new_id
```

```
# -------------------- DISPLAY THE INPUT, PROCESS, AND OUTPUT --------------------
print('=' * 70)
print('BEGINNER TRADING ASSISTANT')
print('Educational demonstration only - not financial advice')
print('=' * 70)

print('\nINPUT KNOWLEDGE BASE')
for category, category_facts in facts.items():
    print(f'{category.title()}:')
    for name, value in category_facts.items():
        print(f"  {name.replace('_', ' ').title()}: {value}")

print('\nRULE-BASED REASONING ACTIONS')
rbr_actions = run_inference_engine(facts)
for number, action in enumerate(rbr_actions, start=1):
    print(f'{number}. {action}')

new_problem = build_new_problem(facts)
ranking = retrieve_most_similar(case_base, new_problem)
print('\nCASE-BASED REASONING - RETRIEVE')
print(f'New problem: {new_problem}')
for rank, (score, case) in enumerate(ranking, start=1):
    print(f"{rank}. {case['case_id']} - similarity: {score * 100:.0f}%")

best_score, best_case = ranking[0]
reused_solution = reuse_solution(best_case)
print('\nCASE-BASED REASONING - REUSE')
print(f"Retrieved case: {best_case['case_id']} ({best_score * 100:.0f}% similar)")
print(f'Reused solution: {reused_solution}')

# A student may replace this text or leave it blank to keep the solution unchanged.
manual_revision = (
    'Keep the asset on a watchlist. If further research supports it, practice the plan '
    'with paper trading before considering a very small staged purchase.'
)
final_solution = revise_solution(reused_solution, manual_revision)
print('\nCASE-BASED REASONING - REVISE')
print(f'Final solution: {final_solution}')

saved_case_id = retain_case(case_base, new_problem, final_solution)
print('\nCASE-BASED REASONING - RETAIN')
print(f'New case saved: {saved_case_id}')
print(f'Updated case count: {len(case_base)}')
print('Updated case base:')
for case in case_base:
    print(f"  {case['case_id']}: {case['solution']}")
```

Program Output

```
======================================================================
BEGINNER TRADING ASSISTANT
Educational demonstration only - not financial advice
======================================================================

INPUT KNOWLEDGE BASE
Market:
  Symbol: DEMO
  Current Price: 105.0
  Moving Average: 100.0
  Rsi: 42
  Trend: up
  Volatility: medium
Investor:
  Risk Tolerance: low
  Experience: beginner
  Has Emergency Fund: True
Portfolio:
  Owns Asset: False
  Allocation Percent: 0
  Cash Available: 5000.0

RULE-BASED REASONING ACTIONS
1. Positive trend detected: add the asset to a watchlist; do not rush to buy.
2. RSI is neutral: use other evidence before making a decision.
3. Beginner rule: if proceeding after research, use only a small amount and define an exit plan.
```

```
CASE-BASED REASONING - RETRIEVE
New problem: {'trend': 'up', 'rsi_band': 'neutral', 'volatility': 'medium', 'risk_tolerance':
'low', 'experience': 'beginner'}
1. TC-001 - similarity: 100%
2. TC-003 - similarity: 55%
3. TC-004 - similarity: 50%
4. TC-002 - similarity: 0%

CASE-BASED REASONING - REUSE
Retrieved case: TC-001 (100% similar)
Reused solution: Watch the asset and, only after more research, consider a very small staged
purchase.

CASE-BASED REASONING - REVISE
Final solution: Keep the asset on a watchlist. If further research supports it, practice the
plan with paper trading before considering a very small staged purchase.

CASE-BASED REASONING - RETAIN
New case saved: TC-005
Updated case count: 5
Updated case base:
  TC-001: Watch the asset and, only after more research, consider a very small staged purchase.
  TC-002: Wait for trend confirmation because an oversold reading can remain weak during a
decline.
  TC-003: Avoid entering while momentum is overbought and volatility exceeds the investor
profile.
  TC-004: Continue observing and practice with a paper-trading plan before committing money.
  TC-005: Keep the asset on a watchlist. If further research supports it, practice the plan
with paper trading before considering a very small staged purchase.
```